C++量化核心编程笔试题

一、手写共享指针（SmartPtr）

题目描述

手动实现一个简化版的C++共享指针（`SharedPtr`），模拟`std::shared_ptr`的核心逻辑，重点考察引用计数、内存管理和拷贝/移动语义。

要求

1. 模板类设计：支持任意类型的指针管理；
2. 核心功能：
   - 构造函数（接收原始指针）、析构函数；
   - 拷贝构造、拷贝赋值运算符（引用计数+1）；
   - 移动构造、移动赋值运算符（转移所有权，不增加引用计数）；
   - 重载`*`和`->`运算符，支持解引用；
   - 提供`get()`方法返回原始指针；
   - 提供`use_count()`方法返回当前引用计数；
   - 提供`reset()`方法重置指针（引用计数-1，计数为0时释放内存）；
3. 边界处理：
   - 禁止空指针的非法操作；
   - 保证多指针指向同一对象时，引用计数准确；
   - 析构时仅当引用计数为0时释放对象内存和引用计数内存；
4. 不依赖STL的智能指针（纯手动实现）。

二、单例模式

题目描述

量化回测系统中，配置管理器（`ConfigManager`）需要全局唯一实例，用于加载和管理回测参数（如初始资金、手续费率、滑点等）。请实现一个线程安全的单例模式，并满足以下要求：

要求

1. 单例模式类型：实现**懒汉式单例**（首次调用时初始化实例），且保证线程安全。

三、限价订单簿撮合引擎（核心撮合逻辑）

题目描述

实现量化回测核心的**限价订单簿撮合引擎**（`OrderBookMatcher`），遵循“价格优先、时间优先”原则完成订单撮合，是回测系统的核心模块。

前置定义

```cpp
#include <cstdint>
#include <string>
#include <vector>

// 订单类型
enum class OrderType {
    BUY,  // 买单（以≤委托价买入）
    SELL  // 卖单（以≥委托价卖出）
};

// 订单结构体
struct Order {
    uint64_t order_id;      // 唯一订单ID
    OrderType type;         // 订单类型
    double price;           // 委托价格
    uint64_t quantity;      // 委托总数量
    uint64_t timestamp;     // 委托时间戳（毫秒级）
    uint64_t remain_qty;    // 剩余未成交数量

    Order(uint64_t id, OrderType t, double p, uint64_t q, uint64_t ts)
        : order_id(id), type(t), price(p), quantity(q), timestamp(ts), remain_qty(q) {}
};

// 成交记录
struct Trade {
    uint64_t trade_id;      // 自增成交ID
    uint64_t buy_order_id;  // 买单ID
    uint64_t sell_order_id; // 卖单ID
    double price;           // 成交价格（以对手方价格为准）
    uint64_t quantity;      // 成交数量
    uint64_t timestamp;     // 成交时间戳

    Trade(uint64_t tid, uint64_t bid, uint64_t sid, double p, uint64_t q, uint64_t ts)
        : trade_id(tid), buy_order_id(bid), sell_order_id(sid), price(p), quantity(q), timestamp(ts) {}
};
```

核心要求

1. 实现 `OrderBookMatcher` 类，包含以下核心接口：
    - `bool add_order(const Order& order)`：添加订单；
    - `std::vector<Trade> match()`：执行撮合逻辑，返回本次撮合产生的所有成交记录；

- `std::vector<Order> get_buy_book()`：获取当前买单簿（符合：价格优先、时间优先）；
- `std::vector<Order> get_sell_book()`：获取当前卖单簿（符合：价格优先、时间优先）；

2. 撮合规则（量化回测标准规则）：
   - 价格优先：买单优先匹配最低价格的卖单，卖单优先匹配最高价格的买单；
   - 时间优先：同价格订单，时间戳小的优先撮合；
   - 成交价格：以**对手方委托价**为准（如买单10元买入，匹配到卖单9.9元，则成交价为9.9元）；
   - 部分成交：订单剩余数量需更新，未成交部分保留在订单簿中；
3. 编写测试用例：覆盖以下场景：
   - 买单与卖单完全成交；
   - 买单与卖单部分成交（剩余数量留在订单簿）；
   - 多笔同价格订单按时间优先撮合；
   - 无匹配对手单（订单直接进入订单簿）。

四、多DAG管理与执行调度

题目描述

实现一个通用的**多DAG调度器**（MultiDAGScheduler），支持管理多个独立的有向无环图（DAG），并按规则完成所有DAG的拓扑排序与执行顺序规划。

前置定义

```cpp
#include <cstdint>
#include <vector>
#include <unordered_map>
#include <unordered_set>
#include <string>

// 单个节点（仅保留唯一标识）
struct Node {
    uint64_t id;  // 节点唯一ID
    Node(uint64_t node_id) : id(node_id) {}
};

// 单个DAG的元信息（区分不同DAG）
struct DAGMeta {
    std::string dag_name;  // DAG名称（如"因子A计算"、"因子B计算"）
    uint64_t dag_id;       // DAG唯一ID
    DAGMeta(uint64_t id, const std::string& name) : dag_id(id), dag_name(name) {}
};
```

核心要求

第一部分：基础单DAG能力

实现 `MultiDAGScheduler` 类的基础接口（管理单个DAG）：

1. `bool add_dag(const DAGMeta& dag_meta)`：添加一个新的空DAG（DAG ID重复返回false，合法返回true）；
2. `bool add_node_to_dag(uint64_t dag_id, const Node& node)`：向指定DAG添加节点（DAG不存在/节点ID重复返回false）；
3. `bool add_edge_to_dag(uint64_t dag_id, uint64_t from_node_id, uint64_t to_node_id)`：向指定DAG添加有向边（DAG/节点不存在/重复加边返回false）；
4. `bool is_dag_valid(uint64_t dag_id) const`：校验指定DAG是否为合法无环图（无环返回true，有环/不存在返回false）；
5. `std::vector<uint64_t> get_dag_topological_order(uint64_t dag_id) const`：返回单个DAG的拓扑排序（节点ID序列），非合法DAG返回空vector。

第二部分：多DAG调度核心

补充以下接口，实现多个DAG的执行规划：

6. `bool set_dag_dependency(uint64_t parent_dag_id, uint64_t child_dag_id)`：设置DAG间的依赖关系（父DAG执行完成后，子DAG才能执行）；

   要求：若DAG不存在/形成DAG间循环依赖（如A→B、B→A），返回false；合法依赖返回true；

7. `std::vector<std::vector<uint64_t>> get_global_execution_plan() const`：生成全局执行计划；

   规则（核心）：

- 执行计划是“批次化”的二维数组：外层vector表示执行批次（按顺序执行），内层vector表示当前批次可**并行执行**的DAG ID；
- 批次规则：

  ✅ 无依赖的DAG归为同一批次（可并行执行）；

  ✅ 有依赖的DAG：父DAG所在批次执行完成后，子DAG才能进入下一批次；

  ✅ 每个DAG内部的节点，仍按自身拓扑排序执行（本接口仅规划DAG间的批次，不关注节点内部顺序）；
- 非法场景：若任意DAG本身不合法（有环），或DAG间存在循环依赖，返回空vector；

边界规

1. 所有DAG本身必须是合法无环图，否则全局执行计划返回空；
2. DAG间依赖仅支持“一对一”，无需处理传递依赖（如A→B、B→C，无需自动识别A→C，仅需显式设置）；
3. 无任何DAG间依赖时，所有合法DAG归为同一批次（可并行）；
4. 单个DAG内部的节点，仅需保证拓扑顺序，无需在本接口中体现执行批次。

示例说明

- 场景1（无DAG间依赖）：

  添加3个合法DAG（ID：1、2、3），无set_dag_dependency；

  全局执行计划返回：`[[1,2,3]]`（3个DAG同一批次并行执行）。
- 场景2（有DAG间依赖）：

  添加4个合法DAG（ID：1、2、3、4）；

  设置依赖：1→2、1→3、3→4；

  全局执行计划返回：`[[1], [2,3], [4]]`；

  解释：

  ✅ 批次1：执行DAG1（无前置依赖）；

✅ 批次2：DAG2（依赖1）、DAG3（依赖1）并行执行；

✅ 批次3：DAG4（依赖3）执行。

- 场景3（DAG间循环依赖）：
  添加DAG1、DAG2，设置依赖1→2、2→1；
  get_global_execution_plan()返回空vector。

代码要求

1. 符合C++11及以上标准，语法无错误；
2. 合理使用STL容器；
3. 编写main函数测试，覆盖以下核心场景：
   ① 添加多个独立合法DAG，验证全局执行计划为单批次；
   ② 添加DAG间依赖（如1→2、1→3），验证执行计划为多批次；
   ③ 添加DAG间循环依赖，验证执行计划返回空；
   ④ 单个DAG内部有环，验证全局执行计划返回空。